

Modul Praktikum Flutter Fundamental

Pertemuan 4: Mengambil Data dari REST API (HTTP, Async, dan FutureBuilder)

Durasi	Level	Prasyarat
120–150 menit	Pemula–Menengah	Pertemuan 1–3 (widget, ListView, navigasi, form, state)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Menjelaskan pemrograman asinkron di Dart: `Future`, `async`, dan `await`.
2. Mengambil data dari REST API memakai paket `http` dan menguraikan JSON.
3. Membuat class model dengan konstruktor `fromJson`.
4. Menampilkan data asinkron dengan `FutureBuilder` lengkap dengan status *loading*, *error*, dan *data*.
5. Menangani kegagalan jaringan dan menyediakan tombol coba lagi.

2. Alat dan Bahan

- Flutter SDK, editor, dan emulator/perangkat dari pertemuan sebelumnya
- Proyek baru: `flutter create praktikum_4`
- Koneksi internet (wajib). API yang dipakai: **JSONPlaceholder** (jsonplaceholder.typicode.com), API publik gratis untuk latihan
- Cadangan: karena API publik dapat sewaktu-waktu lambat atau tidak tersedia, siapkan kesepakatan dengan dosen/asisten untuk mode offline (lihat Troubleshooting)

3. Teori Singkat

Asinkron. Mengambil data dari internet membutuhkan waktu. Agar UI tidak membeku, Dart memakai `Future`: objek yang berisi hasil yang *akan* tersedia nanti. Kata kunci `await` menunggu hasil tanpa memblokir UI, dan hanya boleh dipakai di dalam fungsi bertanda `async`.

REST API dan JSON. Aplikasi mengirim permintaan HTTP (misalnya GET) ke sebuah URL dan menerima respons berformat JSON. Kode status 200 berarti sukses; 404 berarti tidak ditemukan; 500 berarti kesalahan server.

Konsep	Fungsi
<code>http.get(uri)</code>	Mengirim permintaan GET, mengembalikan <code>Future<Response></code>
<code>jsonDecode(body)</code>	Mengubah teks JSON menjadi <code>Map</code> / <code>List</code> Dart
<code>fromJson</code>	Konstruktor factory yang mengubah <code>Map</code> menjadi objek model
<code>FutureBuilder</code>	Widget yang membangun UI berdasarkan status sebuah <code>Future</code>
<code>snapshot</code>	Berisi status (<code>connectionState</code>), data (<code>data</code>), dan error (<code>error</code>)

4. Langkah Praktikum

Bagian A: Mengenal Future dan FutureBuilder (tanpa internet)

Ganti isi `lib/main.dart` dengan kode berikut. Fungsi `ambilSalam()` mensimulasikan pengambilan data yang butuh 2 detik.

```
import 'package:flutter/material.dart';
void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 4',
      theme: ThemeData(colorSchemeSeed: Colors.blue, useMaterial3: true),
      home: const SalamPage(),
    );
  }
}

Future<String> ambilSalam() async {
  await Future.delayed(const Duration(seconds: 2));
  return 'Halo dari masa depan!';
}

class SalamPage extends StatefulWidget {
  const SalamPage({super.key});

  @override
  State<SalamPage> createState() => _SalamPageState();
}

class _SalamPageState extends State<SalamPage> {
  late Future<String> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilSalam();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Demo Future')),
      body: Center(
        child: FutureBuilder<String>(
          future: _future,
          builder: (context, snapshot) {
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator();
            }
            if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');
            }
            return Text(snapshot.data!, style: const TextStyle(fontSize: 24));
          },
        ),
      ),
    );
  }
}
```

Checkpoint: selama 2 detik tampil indikator berputar, lalu berganti menjadi teks. **Penting:** `Future` dibuat di `initState()`, bukan di dalam `build()`, agar tidak diulang setiap kali layar dibangun ulang.

Bagian B: Menyiapkan Paket HTTP

Jalankan di terminal pada folder proyek:

```
flutter pub add http
```

Agar aplikasi *release* Android dapat mengakses internet, tambahkan izin berikut di `android/app/src/main/AndroidManifest.xml`, tepat di atas tag `<application` (mode debug biasanya sudah memilikinya):

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Catatan: bila menjalankan di macOS desktop, izin jaringan keluar (`com.apple.security.network.client`) juga perlu diaktifkan pada berkas entitlements.

Bagian C: Model dan Fungsi Pengambil Data

Struktur data dari `/users` berupa daftar objek JSON dengan kolom seperti `id`, `name`, `email`, `phone`, dan `website`. Tambahkan kode berikut di bawah `import` pada `main.dart`, dan tambahkan dua import baru di bagian paling atas:

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class Pengguna {
  final int id;
  final String name;
  final String email;
  final String phone;
  final String website;

  const Pengguna({
    required this.id,
    required this.name,
    required this.email,
    required this.phone,
    required this.website,
  });

  factory Pengguna.fromJson(Map<String, dynamic> json) {
    return Pengguna(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
      phone: json['phone'] as String,
      website: json['website'] as String,
    );
  }
}

Future<List<Pengguna>> ambilPengguna() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/users');
  final response = await http.get(uri).timeout(const Duration(seconds: 10));

  if (response.statusCode != 200) {
    throw Exception('Gagal memuat data (kode ${response.statusCode})');
  }

  final List<dynamic> data = jsonDecode(response.body);
  return data
    .map((e) => Pengguna.fromJson(e as Map<String, dynamic>))
    .toList();
}
```

Checkpoint: kode tidak ada galat kompilasi. Fungsi `ambilPegguna()` melempar `Exception` bila status bukan 200; galat itu nanti ditangkap oleh `snapshot.error`.

Bagian D: Menampilkan Data dengan FutureBuilder

Tambahkan halaman berikut, lalu ubah `home`: menjadi `const PenggunaPage()`. Halaman ini menangani tiga keadaan: memuat, galat (dengan tombol coba lagi), dan data.

```
class PenggunaPage extends StatefulWidget {
  const PenggunaPage({super.key});

  @override
  State<PenggunaPage> createState() => _PenggunaPageState();
}

class _PenggunaPageState extends State<PenggunaPage> {
  late Future<List<Pengguna>> _future;

  @override
  void initState() {
    super.initState();
    _future = ambilPegguna();
  }

  void _muatUlang() {
    setState(() {
      _future = ambilPegguna();
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Pengguna'),
        actions: [
          IconButton(icon: const Icon(Icons.refresh), onPressed: _muatUlang),
        ],
      ),
      body: FutureBuilder<List<Pengguna>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  mainAxisAlignment: MainAxisAlignment.min,
                  children: [
                    const Icon(Icons.error_outline, size: 48, color: Colors.red),
                    const SizedBox(height: 8),
                    Text(
                      'Terjadi kesalahan:\n${snapshot.error}',
                      textAlign: TextAlign.center,
                    ),
                    const SizedBox(height: 12),
                    ElevatedButton(
                      onPressed: _muatUlang,
                      child: const Text('Coba lagi'),
                    ),
                  ],
                ),
              ),
            );
          }
          final data = snapshot.data!;
```

```

return ListView.builder(
  itemCount: data.length,
  itemBuilder: (context, i) {
    final p = data[i];
    return ListTile(
      leading: CircleAvatar(child: Text(p.name[0])),
      title: Text(p.name),
      subtitle: Text(p.email),
      trailing: const Icon(Icons.chevron_right),
      onTap: () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => DetailPenggunaPage(pengguna: p),
          ),
        );
      },
    );
  },
);

```

Checkpoint: 10 pengguna tampil setelah indikator loading singkat. Tombol refresh di AppBar memuat ulang data.

Bagian E: Halaman Detail

```

class DetailPenggunaPage extends StatelessWidget {
  final Pengguna pengguna;
  const DetailPenggunaPage({super.key, required this.pengguna});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(pengguna.name)),
      body: ListView(
        children: [
          ListTile(
            leading: const Icon(Icons.email),
            title: Text(pengguna.email),
          ),
          ListTile(
            leading: const Icon(Icons.phone),
            title: Text(pengguna.phone),
          ),
          ListTile(
            leading: const Icon(Icons.language),
            title: Text(pengguna.website),
          ),
        ],
      ),
    );
  }
}

```

Checkpoint: mengetuk pengguna membuka halaman detail berisi email, telepon, dan situs web.

Bagian F: Percobaan Penanganan Galat

Lakukan dua percobaan berikut dan catat tampilan yang muncul untuk laporan:

1. Ubah URL menjadi `/userz` (halaman tidak ada). Amati pesan galat dengan kode 404, lalu kembalikan URL yang benar dan tekan *Coba lagi*.

2. Matikan koneksi internet perangkat/emulator lalu tekan tombol refresh. Amati pesan galat jaringan, nyalakan internet kembali, dan tekan *Coba lagi*.

5. Latihan Mandiri

1. Tambahkan kolom `username` dan kota (data bersarang: `json['address']['city']`) pada model dan tampilkan di halaman detail.
2. Bungkus `ListView` dengan `RefreshIndicator` agar dapat ditarik ke bawah untuk memuat ulang. Petunjuk: `onRefresh` wajib mengembalikan `Future`, misalnya panggil `_muatUlang()` lalu `await _future` di dalam `try/catch` agar galat tetap ditampilkan oleh `FutureBuilder`.
3. Bila daftar kosong, tampilkan teks "Tidak ada data".
4. Tambahkan `Text` di `AppBar` yang menampilkan jumlah pengguna setelah data berhasil dimuat. Petunjuk: `title AppBar` dapat memakai `FutureBuilder` kedua dengan `_future` yang sama.

6. Tugas

Buat aplikasi **Daftar Postingan** menggunakan endpoint `/posts` dan `/posts/{id}/comments` dengan ketentuan:

- Halaman utama menampilkan daftar postingan (judul dan potongan isi) dengan `FutureBuilder`.
- Mengetuk postingan membuka halaman detail yang menampilkan isi lengkap dan **daftar komentar** yang diambil dari endpoint terpisah (`Future` kedua).
- Model `Post` dan `Komentar` dengan `fromJson`; kode pengambil data dipisah dari kode UI.
- Menangani status loading dan galat pada kedua halaman, termasuk tombol *coba lagi*.
- Kumpulkan: tangkapan layar (daftar, detail, dan tampilan galat) serta berkas `main.dart` (atau tautan repositori).

7. Rubrik Penilaian

Komponen	Bobot
Bagian A–E berjalan (checkpoint)	30%
Percobaan galat (Bagian F) dan latihan mandiri	20%
Tugas Daftar Postingan (fungsi, penanganan loading/galat)	40%
Kerapian kode dan pemisahan model/layanan/UI	10%

8. Pertanyaan Refleksi

1. Apa yang terjadi bila `Future` dibuat di dalam `build()` dan bukan di `initState()`? Mengapa?
2. Apa perbedaan `snapshot.hasError` dengan memeriksa `response.statusCode`? Mengapa keduanya diperlukan?
3. Mengapa data JSON sebaiknya diubah menjadi class model, bukan dipakai langsung sebagai `Map`?
4. Mengapa UI wajib menyediakan status loading dan error, bukan hanya status data?

9. Troubleshooting Umum

Masalah	Solusi
<code>SocketException</code> / Failed host lookup	Periksa koneksi internet emulator; untuk build release pastikan izin INTERNET sudah ada
Package http tidak ditemukan	Jalankan <code>flutter pub get</code> lalu restart aplikasi (bukan hot reload)
Data dimuat ulang setiap layar berubah	Pindahkan pembuatan <code>Future</code> ke <code>initState()</code>
Error: type 'Null' is not a subtype	Nama kunci JSON tidak cocok atau bernilai null; cek respons asli di browser
API publik lambat/tidak tersedia	Gunakan tunggu lebih lama, coba lagi nanti, atau minta berkas JSON cadangan dari asisten (dibaca dengan <code>rootBundle</code>)

10. Referensi

- Mengambil data dari internet: docs.flutter.dev/cookbook/networking/fetch-data
- Async dan await di Dart: dart.dev/codelabs/async-await
- Paket http: pub.dev/packages/http
- JSONPlaceholder: jsonplaceholder.typicode.com